

[linear.algebra]R1

A proposal to add linear algebra support to the C++ standard library

Working Draft, 2019-03-10

This version:

https://github.com/BobSteagall/wg21/blob/master/linear_algebra/papers/P1385R1.html

Latest published version:

<http://wg21.link/P1385>

Authors:

[Guy Davidson](#)

[Bob Steagall](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

SG14, LEWG

Previous version:

<http://wg21.link/P1385R0>

Strawman implementation:

https://github.com/BobSteagall/wg21/tree/master/linear_algebra/code

Abstract

This document proposes an interface specification for a set of fundamental linear algebra facilities in the standard C++ library.

Table of Contents

1 Introduction

2 Goals

3 Definitions

3.1 Mathematical Terms

3.2 Terms Pertaining to C++ Types

3.3 Overloaded Terms

3.3.1 Matrix

3.3.2 Vector

3.3.3 Dimension

3.3.4 Rank

4 Scope

4.1 Functionality

4.2 Considered but Excluded

5 Design Aspects

5.1 Memory Source

5.2 Addressing Model

5.3 Memory Ownership

5.4 Capacity and Resizability

5.5 Element Layout

5.6 Element Access and Indexing

5.7 Element Type

5.8 Mixed-Element-Type Expressions

5.9 Mixed-Engine Expressions

5.10 Arithmetic Customization

5.11 Linear Algebra and `constexpr`

6 Interface Design Description

6.1 Overview

6.2 Element Promotion Helper Traits

6.3 Element Promotion Traits

6.4 Engine Types

6.4.1 Fixed-Size Engine

6.4.2 Dynamically-Resizable Engine

- 6.4.3 Transpose Engine
- 6.5 Engine Promotion Traits
 - 6.5.1 For Negation
 - 6.5.2 For Addition
 - 6.5.3 For Subtraction
 - 6.5.4 For Multiplication
- 6.6 Mathematical Types
 - 6.6.1 Helper Traits
 - 6.6.2 Column Vector
 - 6.6.3 Row Vector
 - 6.6.4 Matrix
- 6.7 Matrix Arithmetic Traits
 - 6.7.1 Negation Traits
 - 6.7.2 Addition Traits
 - 6.7.3 Subtraction Traits
 - 6.7.4 Multiplication Traits
- 6.8 Operator Traits
- 6.9 Operator Traits Promotion
- 6.10 Arithmetic Operators
 - 6.10.1 Negation Operator
 - 6.10.2 Addition Operator
 - 6.10.3 Subtraction Operator
 - 6.10.4 Multiplication Operator
- 6.11 Type Aliases
- 6.12 Expression Results

7 How Does it Work?

- 7.1 In More Detail
 - 7.1.1 Calling an Operator
 - 7.1.2 Selecting the Operator Traits
 - 7.1.3 Selecting the Multiplication Traits
 - 7.1.4 Selecting the Engine Type
 - 7.1.5 The Fully Instantiated Multiplication Traits

8 Customization

- 8.1 Integrating a New Element Type
- 8.2 Integrating a New Engine Type
- 8.3 Customizing an Arithmetic Operation

9 Future Directions

- 9.1 Kona Feedback
 - 9.1.1 LEWG(I) Polls and Feedback (Wednesday 2019-02-20)
 - 9.1.2 Feedback From Joint SG14/SG19 Session (Friday 2019-02-22)
 - 9.1.3 Other Suggestions Gathered Received at Kona

Revision History

Version	Description
R0	Initial version for pre-Kona mailing
D1	Update for presentation at Kona; includes operator traits, some customization examples, and several typo and readability fixes.
R1	Update for post-Kona mailing; includes feedback from LEWG(I) and joint SG14/SG19 session.

§ 1. Introduction

Linear algebra is a mathematical discipline of ever-increasing importance, with direct application to a wide variety of problem domains, such as signal processing, computer graphics, medical imaging, scientific simulations, machine learning, analytics, financial modeling, and high-performance computing. And yet, despite the relevance of linear algebra to so many aspects of modern computing, the C++ standard library does not include a set of linear algebra facilities. This paper proposes to remedy this deficit for C++23.

This paper should be read after P1166, in which we describe a high-level set of expectations for what a linear algebra library should contain.

§ 2. Goals

We expect that typical users of a standard linear algebra library are likely to value two features above all else: ease-of-use (including expressiveness), and high performance out of the box. This set of users will expect the ability to compose arithmetical expressions of linear algebra objects similar to what

one might find in a textbook; indeed, this has been deemed a "must-have" feature by several participants in recent SG14 Linear Algebra SIG conference calls. And for a given arithmetical expression, they will expect run-time computational performance that is close to what they could obtain with an equivalent sequence of function calls to a more "traditional" linear algebra library, such as *LAPCK*, *Blaze*, *Eigen*, etc.

There also exists a set of linear algebra “super-users” who will value most highly a third feature – the ability to customize underlying infrastructure in order to maximize performance for specific problems and computing platforms. These users seek the highest possible run-time performance, and to achieve it, require the ability to customize any and every portion of the library’s computational infrastructure.

With these high-level user requirements in mind, in this paper we propose an interface specification intended to achieve the following goals:

1. To provide a set of vocabulary types for representing the mathematical objects and operations that are relevant to linear algebra;
2. To provide a public interface for linear algebra operations that is intuitive, teachable, and mimics the expressiveness of mathematical notation to the greatest *reasonable* extent;
3. To exhibit out-of-the-box performance in the neighborhood of that of that exhibited by an equivalent sequence of function calls to a more traditional linear algebra library, such as *LAPACK*, *Blaze*, *Eigen*, etc.;
4. To provide a set of building blocks that manage the source, ownership, lifetime, layout, and access of the memory required to represent the linear algebra vocabulary types, with the requirement that some of these building blocks are also suitable for (eventually) representing other interesting mathematical entities, such as quaternions, octonions, and tensors;
5. To provide straightforward facilities and techniques for customization that enable users to optimize performance for their specific problem domain on their specific hardware; and,
6. To provide a *reasonable* level of granularity for customization so that developers only have to implement a minimum set of types and functions to integrate their performance enhancements with the rest of the linear algebra facilities described here.

§ 3. Definitions

When discussing linear algebra and related topics for a proposal such as this, it is important to note that there are several overloaded terms (such as *matrix*, *vector*, *dimension*, and *rank*) which must be defined and disambiguated if such discussions are to be productive. These terms have specific meanings in mathematics, as well as different, but confusingly similar, meanings to C++ programmers.

In the following sections we provide definitions for relevant mathematical concepts, C++ type design concepts, and describe how this proposal employs those overloaded terms in various contexts.

§ 3.1. Mathematical Terms

In order to facilitate subsequent discussion, we first provide the following *informal* set of definitions for important mathematical concepts:

1. A **vector space** is a collection of **vectors**, where vectors are objects that may be added together and multiplied by scalars. Euclidean vectors are an example of a vector space, typically used to represent displacements, as well as physical quantities such as force or momentum. Linear algebra is concerned primarily with the study of vector spaces.
2. The **dimension** of a vector space is the minimum number of coordinates required to specify any point within the space.
3. A **matrix** is a rectangular array of numbers, symbols, or expressions, arranged in rows and columns. A matrix having m rows and n columns is said to have size $m \times n$. Although matrices can be used solve systems of simultaneous linear equations, they are most commonly used to represent linear transformations, solve linear least squares problems, and to explore and/or manipulate the properties of vector spaces.
4. The **rank** of a matrix is the dimension of the vector space spanned by its columns, which is equal to the dimension of the vector space spanned by its rows. The rank is also equal to the maximum number of linearly-independent columns and rows.
5. An **element** of a matrix is an individual member (number, symbol, expression) of the rectangular array comprising the matrix, lying at the intersection of a single row and a single column. In traditional mathematical notation, row and column indexing is 1-based, where rows are indexed from 1 to m and columns are indexed from 1 to n . Given some matrix A , element a_{11} refers to the element in the upper left-hand corner of the array and element a_{mn} refers to the element in the lower right-hand corner.
6. A **row vector** is a matrix containing a single row; in other words, a matrix of size $1 \times n$. In many applications of linear algebra, row vectors represent spatial vectors.
7. A **column vector** is a matrix containing a single column; in other words, a matrix of size $m \times 1$. In many applications of linear algebra, column vectors represent spatial vectors.
8. **Element transforms** are non-arithmetical operations that modify the relative positions of elements in a matrix, such as transpose, column exchange, and row exchange.

9. **Element arithmetic** refers to arithmetical operations that read or modify the values of individual elements independently of other elements, such as assigning a value to a specific element or multiplying a row by some value.
10. **Matrix arithmetic** refers to the assignment, addition, subtraction, negation, multiplication, and determinant operations defined for matrices, row vectors, and column vectors as wholes.
11. A **rectangular matrix** is a matrix requiring a full $m \times n$ representation; that is, a matrix not possessing a special form, such as identity, triangular, band, etc.
12. The **identity matrix** is a square matrix where all elements on the diagonal are equal to one and all off-diagonal elements are equal to zero.
13. A **triangular matrix** is a matrix where all elements above or below the diagonal are zero; those with non-zero elements above the diagonal are called *upper triangular*, while those with non-zero elements below the diagonal are called *lower triangular*.
14. A **band matrix** is a sparse matrix whose non-zero entries are confined to a diagonal band, lying on the main diagonal and zero or more diagonals on either side.
15. **Decompositions** are complex sequences of arithmetic operations, element arithmetic, and element transforms performed upon a matrix that expose important mathematical properties of that matrix. Several types of decomposition are often performed in solving least-squares problems.
16. **Eigen-decompositions** are decompositions performed upon a symmetric matrix in order to compute the eigenvalues and eigenvectors of that matrix; this is often performed when solving problems involving linear dynamic systems.

§ 3.2. Terms Pertaining to C++ Types

The following are terms used in this proposal that describe various aspects of how the mathematical concepts described above in Section 3.1 might be implemented:

1. An **array** is a data structure representing an indexable collection of objects (elements) such that each element is identified by at least one index. An array is said to be *one-dimensional* array if its elements are accessible with a single index; a *multi-dimensional* array is an array for which more than one index is required to access its elements.
2. The **dimension** of an array refers to the number of indices required to access an element of that array. The **rank** of an array is a synonym for its dimension.
3. This proposal uses the term **MathObj** to refer generically to one of the C++ types described herein representing matrices, row vectors, and column vectors. These are the public-facing

facilities developers will use in their code.

4. An **engine** is an implementation type that manages the resources associated with a *MathObj* instance. This includes, at a minimum, the storage-related aspects of, and access to, the elements of a *MathObj*. It could also include execution-related aspects, such as an execution context. In this proposal, an engine object is a private member of a *MathObj*. Other than as a template parameter, engines are not part of a *MathObj*'s public interface.
5. The adjective **dense** refers to a *MathObj* representation where storage is allocated for every element.
6. The adjective **sparse** refers to a *MathObj* representation where storage is allocated only for non-zero elements;
7. **Storage** is used by this proposal as a synonym for memory.
8. **Traits** refers to a stateless class template that provides some set of services, normalizing those services over its set of template parameters.
9. **Row size** and **column size** refer to the number of rows and columns, respectively, that a *MathObj* represents, which must be less than or equal to its row and column capacities, defined below.
10. **Row capacity** and **column capacity** refer to the maximum number of rows and columns, respectively, that a *MathObj* can possibly represent.
11. **Fixed-size** (FS) refers to an engine type whose row and column sizes are fixed at instantiation time and constant thereafter.
12. **Fixed-capacity** (FC) refers to an engine type whose row and column capacities are fixed at instantiation time and constant thereafter.
13. **Dynamically re-sizable** (DR) refers to an engine type whose row and column sizes and capacities may be changed at run time.

§ 3.3. Overloaded Terms

This section describes how we use certain overloaded terms in this proposal and in future works.

§ 3.3.1. Matrix

The term *matrix* is frequently used by C++ programmers to mean a general-purpose array of arbitrary size. For example, one of the authors worked at a company where it was common practice to refer to 4-dimensional arrays as “4-dimensional matrices.”

In this proposal, we use the word *array* only to mean a data structure whose elements are accessible using one or more indices, and which has no invariants pertaining to higher-level or mathematical meaning.

We use *matrix* to mean the mathematical object as defined above in Section 3.1, and `matrix` (in monospaced font) to mean the C++ class template that implements the mathematical object. We sometimes use `MathObj` (in monospaced font) in some of the component interface code and text below to generically refer to a `matrix`, `row_vector`, or `column_vector` object.

§ 3.3.2. Vector

Likewise, many C++ programmers incorrectly use the term *vector* as a synonym for “dynamically resizable array.” This bad habit is exacerbated by the unfortunate naming of `std::vector`.

This proposal uses the term *vector* to mean an element of a vector space, per Section 3.1. Further, *row vector* and *column vector* have the meanings set out in 3.1, while `row_vector` and `column_vector` (in monospaced font) are the C++ class templates implementing those mathematical objects, respectively. We sometimes use `MathObj` (in monospaced font) in some of the component code interface code and test below to generically refer to a `row_vector`, `column_vector`, or `matrix` object.

§ 3.3.3. Dimension

In linear algebra, a vector space V is said to be of *dimension* n , or be *n-dimensional*, if there exist n linearly independent vectors which span V . This is another way of saying that n is the minimum number of coordinates required to specify any point in V . However, in common programming parlance, *dimension* refers to the number of indices used to access an element in an array.

We use the term dimension both ways in this proposal, but try to do so consistently and in a way that is clear from the context. For example, a rotation matrix used by a game engine is two-dimensional data structure composed of three-dimensional row and column vectors. A vector describing an electric field is an example of a one-dimensional data structure that implemented as a three-dimensional column vector.

§ 3.3.4. Rank

The *rank* of a matrix is the dimension of the vector space spanned by its columns (or rows), which corresponds to the maximal number of linearly independent columns (or rows) of that matrix. Rank

also has another meaning in tensor analysis, where it is commonly used as a synonym for a tensor's *order*.

However, rank also has a meaning in computer science where it is used as a synonym for dimension. In the C++ standard at `[meta.unary.prop.query]`, rank is described as the number of dimensions of `T` if `T` names an array, otherwise it is zero.

We avoid using the term *rank* in this proposal in the context of linear algebra, except as a quantity that might result from performing certain decompositions wherein the mathematical rank of a matrix is computed.

§ 4. Scope

We contend that the best approach for standardizing a set of linear algebra components for C++23 will be one that is layered, iterative, and incremental. This paper is quite deliberately a “basic linear algebra-only” proposal; it describes what we believe is a foundational layer providing the minimum set of components and arithmetic operations necessary to provide a reasonable, basic level of functionality.

Higher-level functionality can be specified in terms of the interfaces described here, and we encourage succession papers to explore this possibility.

§ 4.1. Functionality

The foundational layer, as described here, should include the minimal set of types and functions required to perform matrix arithmetic in finite dimensional spaces. This includes:

- Matrix, row vector, and column vector class templates;
- Arithmetic operations for addition, subtraction, negation, and multiplication of matrices and row/column vectors;
- Arithmetic operations for scalar multiplication of matrices and row/column vectors;
- Well-defined facilities for integrating new element types;
- Well-defined facilities for creating and integrating custom engines; and,
- Well-defined facilities for creating and integrating custom arithmetic operations.

§ 4.2. Considered but Excluded

Tensors

There has been a great deal of interest expressed in specifying an interface for general-purpose tensor processing in which linear algebra facilities fall out as a special case. We exclude this idea from this proposal for two reasons. First, given the practical realities of standardization work, the enormous scope of such an effort would very likely delay introduction of linear algebra facilities until C++26 or later.

Second, and more importantly, implementing matrices as derived types or specializations of a general-purpose tensor type is bad type design. Consider the following: a tensor is (informally) an array of mathematical objects (numbers or functions) such that its elements transform according to certain rules under a coordinate system change. In a p -dimensional space, a tensor of rank n will have p^n elements. In particular, a rank-2 tensor in a p -dimensional space may be represented by a $p \times p$ matrix having certain invariants related to coordinate transformation not possessed by all $p \times p$ matrices.

These defining characteristics of a tensor lead us to the crux of the issue: every rank-2 tensor can be represented by a square matrix, but not every square matrix represents a tensor. As one quickly realizes, only a small fraction of all possible matrices are representations of rank-2 tensors.

All of this is a long way of saying that the class invariants governing a matrix type are quite different from those governing a tensor type, and as such, the public interfaces of such types will also differ substantially.

From this we conclude that matrices are not Liskov-substitutable for rank-2 tensors, and therefore as matter of good type design, matrices and tensors should be implemented as distinct types, perhaps with appropriate inter-conversion operations.

This situation is analogous to the age-old object-oriented design question: when designing a group of classes that represent geometric shapes, is a square a kind of rectangle? In other words, should class `square` be publicly derived from class `rectangle`? Mathematically, yes, a square is a rectangle. But from the perspective of good interface design, class `square` is not substitutable for class `rectangle` and is usually best implemented as a distinct type having no IS-A relationship with `rectangle`.

Quaternions and Octonions

There has also been interest expressed in including other useful mathematical objects, such as quaternions and octonions, as part of a standard linear algebra library. Although element storage for these types might be implemented using the engines described in this proposal, quaternions and octonions represent mathematical concepts that are fundamentally different from those of matrices and vectors.

As with tensors, the class invariants and public interfaces for quaternions and octonions would be substantially different from that of the linear algebra components. Liskov substitutability would not be possible, and therefore quaternions and octonions should be implemented as a set of types distinct from the linear algebra types.

§ 5. Design Aspects

The following describe several important aspects of the problem domain affecting the design of the proposed interface. Importantly, these aspects are orthogonal, and are addressable through judicious combinations of template parameters and implementation type design.

§ 5.1. Memory Source

Perhaps the first question to be answered is that of the source of memory in which elements will reside. One can easily imagine multiple sources of memory:

- Elements reside in an external buffer allocated from the global heap.
- Elements reside in an external buffer allocated by a custom allocator and/or specialized heap.
- Elements reside in an external fixed-size buffer that exists independently of the *MathObj*, not allocated from a heap, and which has a lifetime greater than that of the *MathObj*.
- Elements reside in a fixed-size buffer that is a member of the *MathObj* itself.
- Elements reside collectively in a set of buffers distributed across multiple machines.

§ 5.2. Addressing Model

It is also possible that the memory used by a *MathObj* might be addressed using what the standard calls a *pointer-like type*, also known as a *fancy pointer*.

For example, consider an element buffer existing in a shared memory segment managed by a custom allocator. In this case, the allocator might employ a fancy pointer type that performs location-independent addressing based on a segment index and an offset into that segment.

One can also imagine a fancy pointer that is a handle to a memory resource existing somewhere on a network, and addressing operations require first mapping that resource into the local address space, perhaps by copying over the network or by some magic sequence of RPC invocations.

§ 5.3. Memory Ownership

The next important questions pertain to memory ownership. Should the memory in which elements reside be deallocated, and if so, what object is responsible for performing the deallocation?

A *MathObj* might own the memory in which it stores its elements, or it might employ some non-owning view type, like `mdspan`, to manipulate elements owned by some other object.

§ 5.4. Capacity and Resizability

As with `std::string` and `std::vector`, it is occasionally useful for a *MathObj* to have excess storage capacity in order to reduce the number of re-allocations required by anticipated future resizing operations. Some linear algebra libraries, like *LAPACK*, account for the fact that a *MathObj*'s capacity may be different than its size. This capability was of critical importance to the success of one author's prior work in functional MRI image analysis.

In other problem domains, like computer graphics, *MathObjs* are small and always of the same size. In this case, the size and capacity are equal, and there is no need for a *MathObj* to maintain or manage excess capacity.

§ 5.5. Element Layout

There are many ways to arrange the elements of a matrix in memory, the most common in C++ being row-major dense rectangular. In Fortran-based libraries, the two-dimensional arrays used to represent matrices are usually column-major. There are also special arrangements of elements for upper/lower triangular and banded diagonal matrices that are both row-major and column-major. These arrangements of elements have been well-known for many years, and libraries like *LAPACK* in the hands of a knowledgeable user can use them to implement code that is optimal in both time and space.

§ 5.6. Element Access and Indexing

In keeping with the goal of supporting a natural syntax, and in analogy with the indexing operations provided by the random-access standard library containers, it seems reasonable to provide both const and non-const indexing for reading and writing individual elements.

However, support for element indexing raises an important question: should *MathObjs* employ 1-based indexing or 0-based indexing? 1-based indexing is the convention used in mathematical notation (and Fortran), whereas 0-based indexing is “the C++ way.”

§ 5.7. Element Type

C++ supports a relatively narrow range of arithmetic types, lacking direct support for arbitrary precision numbers and fixed-point numbers, among others. Libraries exist to implement these types, and they should not be precluded from use in a standard linear algebra library. It is possible that individual elements of a *MathObj* may allocate memory, and therefore an implementation cannot assume that element types have trivial constructors or destructors.

§ 5.8. Mixed-Element-Type Expressions

In general, when multiple built-in arithmetic types are present in an arithmetical expression, the resulting type will have a precision greater than or equal to that of the type with greatest precision in the expression. In other words, to the greatest reasonable extent, information is preserved.

We contend that a similar principal should apply to expressions involving *MathObjs* where more than one element type is present. Arithmetic operations involving *MathObjs* should, to the greatest reasonable extent, preserve element-wise information.

For example, just as the result of multiplying a `float` by a `double` is a `double`, the result multiplying a matrix-of-`float` by a matrix-of-`double` should be a matrix-of-`double`. We call the process of determining the resulting element type *element promotion*.

§ 5.9. Mixed-Engine Expressions

In analogy with element type, *MathObj* expressions may include mixed storage management strategies, as implemented by their corresponding engine types. For example, consider the case of a fixed-size matrix multiplied by a dynamically-resizable matrix. What is the engine type of the resulting matrix?

Expression involving mixed engine types should not limit the availability of basic arithmetic operations. This means that there should be a mechanism for determining the engine type of the resulting from such expressions. We call the process of determining the resulting engine type *engine promotion*.

We contend that in most cases, the resulting engine type should be at least as "general" as the most "general" of the two engine types. For example, one could make the argument that a dynamically-resizable engine is more general than a fixed-size engine, and therefore the resulting engine type in an expression involving both these engine types should be a dynamically-resizable engine.

§ 5.10. Arithmetic Customization

In pursuit of optimal performance, developers may want to customize specific arithmetic operations, such as matrix-matrix or matrix-vector multiplication. Customization might be based on things like element layout in memory, fixed-size -vs- dynamically resizable, special hardware capabilities, etc.

One such possible optimization is the use of multiple cores, perhaps distributed across a network, to carry out multiplication on very large pairs of matrices, particularly in situations where the operation is used to produce a third matrix rather than modify one of the operands; the matrix multiplication operation is particularly amenable to this approach.

Developers may also wish to make use of SIMD intrinsics to enable parallel evaluation of matrix multiplication. This is common in game development environments where programs are written for very specific platforms, where the make and model of processor is well defined. This would impact on element layout and storage. Such work has already been demonstrated in paper N4454.

It is possible that two operands may be associated with different arithmetic customizations. We call the process of determining which of those two customizations to employ when performing the actual arithmetic operations *operator traits promotion*.

§ 5.11. Linear Algebra and `constexpr`

The fundamental set of operations for linear algebra can all be implemented in terms of a subset of the algorithms defined in the `<algorithm>` header, all of which are marked `constexpr` since C++20. Matrix and vector initialization is of course also possible at compile time.

The arrival of `std::is_constant_evaluated` in C++20 makes it possible to offer a `constexpr` implementation of the operations, allowing customizations to defer to them in `constexpr` evaluations while taking the customization's notionally superior run-time path in alternate situations.

§ 6. Interface Design Description

In this section, we describe the various types, operators, and functions comprising the proposed interface. The reader should note that the descriptions below are by no means ready for wording; rather, they are intended to foster further discussions and refinements, and to serve as a guide for brave souls attempting to build implementations from this specification.

§ 6.1. Overview

At the highest level, the interface is divided into three broad categories:

1. **MathObjs**, each of which provides an interface intended to model a corresponding mathematical abstraction (column vector, row vector, matrix);
2. **Engines**, which are implementation types that manage the resources associated with a *MathObj* instance; and,
3. **Operators**, which provide the desired mathematical syntax and carry out the promised arithmetic.

At a lower level, and behind the scenes, are a number of supporting traits types employed by the operators to determine the return type of the operator and perform the corresponding arithmetic operation. There are several such traits types, which are intended to act as customization points:

- **Element promotion helper traits** determine whether or not a given type `T` is an acceptable *MathObj* element type.
- **Element promotion traits** determine the resulting element type for an arithmetical operation based on two the element types of two input operands. This traits type uses the element promotion helper traits to verify that both input types are in fact acceptable matrix element types.
- **Engine promotion traits** determine the type of an engine resulting from an arithmetical operation. As part of that process, this traits type uses the element promotion traits to determine the element type of the resulting engine.
- **Arithmetic traits** determine the type of a *MathObj* resulting from an arithmetical operation. As part of that process, this traits type uses the engine promotion traits to determine the engine type of the resulting *MathObj*. Having determined the result type, the arithmetic traits also have a member function that carries out the actual computations.
- **Operator traits** specify a collection of four specific arithmetic traits types, one type for each supported arithmetic operation. This traits type is a template parameter to the *MathObj* types, and provides a way to inform an operator of the set of suggested arithmetic traits to be used when deciding how to perform an arithmetic operation.
- **Operator promotion traits** provide the means by which an operator selects the operator traits that will perform the arithmetic. In the case where each operand has the same operator traits, the decision is simple. However, it is possible that the operands may be instantiated with different operator traits types, and in this case, the operator must decide with operator traits type to use in computing its result.

The following sections describe the building blocks in more detail, starting at the lowest level, and working upward in order of dependency.

§ 6.2. Element Promotion Helper Traits

Traits type `is_complex<T>` determines whether its template parameter `T` is of type `std::complex<V>` for some type `V`, where `V` must itself be an arithmetical type as determined by `std::is_arithmetic_v<U>`. Defining what constitutes an arithmetic type can be challenging; our intention is that an arithmetic type is one that models a field, although types that model rings should work as well.

```
template<class T> struct is_complex;
template<class T> constexpr bool is_complex_v = ...;
```

Traits type `is_matrix_element<T>` is used in static assertions to ensure that *MathObj* types are instantiated only with element types representing a field or ring (i.e., arithmetic types, or complex types per above). It uses `is_complex<T>` as appropriate to help make that determination.

```
template<class T> struct is_matrix_element;
template<class T> constexpr bool is_matrix_element_v = ...;
```

§ 6.3. Element Promotion Traits

Element promotion traits are used by the engine promotion traits to determine the resulting element type in a binary expression involving engines with possibly two different element types.

Class template `matrix_element_promotion<T1, T2>` implements a traits type that determines the appropriate result type of an arithmetical operation involving two (possibly different) *MathObj* element types.

```

template<class T1, class T2>
struct matrix_element_promotion
{
    using type = ...;          //- Implementation-defined
};

template<class T1, class T2>
struct matrix_element_promotion<T1, std::complex<T2>>;

template<class T1, class T2>
struct matrix_element_promotion<std::complex<T1>, T2>;

template<class T1, class T2>
struct matrix_element_promotion<std::complex<T1>, std::complex<T2>>;

template<class T1, class T2>
using matrix_element_promotion_t = typename matrix_element_promotion<T1, T2>::type;

```

Note the partial specializations involving `complex<T>` elements. Also, as will be common practice for most of the traits types in this proposal, a parameterized alias for the trait is also defined.

§ 6.4. Engine Types

The over-arching purpose of the engine types is to perform resource management on behalf of an associated *MathObj* instance. At a minimum, all of the engine types provide a basic interface for const element indexing, row and column sizes, and row and column capacities. They also export public type aliases which specify their element type, whether or not they are dense, whether or not they are rectangular, whether or not they are resizable, whether or not their memory layout is row-major, and a 2-tuple for describing sizes and capacities.

One can imagine engines that also manage resources related to execution.

§ 6.4.1. Fixed-Size Engine

Class template `fs_matrix_engine<T, Rows, Cols>` implements a fixed-size, fixed-capacity engine. In addition to the basic engine interface, it provides member functions for mutable element indexing and swapping rows and/or columns.

```

template<class T, size_t Rows, size_t Cols>
class fs_matrix_engine
{
    static_assert(Rows >= 1);
    static_assert(Cols >= 1);

public:
    using element_type    = T;
    using is_dense        = std::true_type;
    using is_rectangular  = std::true_type;
    using is_resizable    = std::false_type;
    using is_row_major    = std::true_type;
    using size_tuple      = std::tuple<size_t, size_t>;

public:
    fs_matrix_engine() = default;
    fs_matrix_engine(fs_matrix_engine&&) = default;
    fs_matrix_engine(fs_matrix_engine const&) = default;

    fs_matrix_engine& operator =(fs_matrix_engine&&) = default;
    fs_matrix_engine& operator =(fs_matrix_engine const&) = default;

    T operator ()(size_t i) const;
    T operator ()(size_t i, size_t j) const;
    T const* data() const noexcept;

    size_t columns() const noexcept;
    size_t rows() const noexcept;
    size_tuple size() const noexcept;

    size_t column_capacity() const noexcept;
    size_t row_capacity() const noexcept;
    size_tuple capacity() const noexcept;

    T& operator ()(size_t i);
    T& operator ()(size_t i, size_t j);
    T* data() noexcept;

    void swap_columns(size_t i, size_t j);
    void swap_rows(size_t i, size_t j);

private:

```

```
    T      ma_elems[Rows*Cols];    //- For exposition; data buffer  
};
```

§ 6.4.2. Dynamically-Resizable Engine

Class template `dr_matrix_engine<T, Alloc>` implements an engine whose sizes and capacities can be changed at runtime. In addition to the basic engine interface, it provides member functions for mutable element indexing, changing size and capacity, and swapping rows and/or columns.

```

template<class T, class Alloc=std::allocator<T>>
class dr_matrix_engine
{
public:
    using element_type    = T;
    using is_dense        = std::true_type;
    using is_rectangular  = std::true_type;
    using is_resizable    = std::true_type;
    using is_row_major    = std::true_type;
    using size_tuple      = std::tuple<size_t, size_t>;

public:
    dr_matrix_engine();
    dr_matrix_engine(dr_matrix_engine&&);
    dr_matrix_engine(dr_matrix_engine const&);
    dr_matrix_engine(size_tuple size);
    dr_matrix_engine(size_t rows, size_t cols);
    dr_matrix_engine(size_tuple size, size_tuple cap);
    dr_matrix_engine(size_t rows, size_t cols, size_t rowcap, size_t colcap);

    dr_matrix_engine& operator =(dr_matrix_engine&&);
    dr_matrix_engine& operator =(dr_matrix_engine const&);

    T operator ()(size_t i) const;
    T operator ()(size_t i, size_t j) const;
    T const* data() const noexcept;

    size_t columns() const noexcept;
    size_t rows() const noexcept;
    size_tuple size() const noexcept;

    size_t column_capacity() const noexcept;
    size_t row_capacity() const noexcept;
    size_tuple capacity() const noexcept;

    T& operator ()(size_t i);
    T& operator ()(size_t i, size_t j);
    T* data() noexcept;

    void reserve(size_tuple cap);
    void reserve(size_t rowcap, size_t colcap);

    void resize(size_tuple size);

```

```

void    resize(size_t rows, size_t cols);
void    resize(size_tuple size, size_tuple cap);
void    resize(size_t rows, size_t cols, size_t rowcap, size_t colcap);

void    swap_columns(size_t i, size_t j);
void    swap_rows(size_t i, size_t j);

private:
    using pointer = typename std::allocator_traits<Alloc>::pointer;

    pointer    mp_elems;          //- For exposition; data buffer
    size_t     m_rows;
    size_t     m_cols;
    size_t     m_rowcap;
    size_t     m_colcap;
};

```

§ 6.4.3. Transpose Engine

Class template `matrix_transpose_engine<Eng>` implements a non-owning, const view type that provides the basic engine interface. Its primary anticipated use case is as the return value of the `t()` member function of the *MathObj* types.

```

template<class Eng>
class matrix_transpose_engine
{
public:
    using engine_type      = Eng;
    using element_type     = typename engine_type::element_type;
    using is_dense         = typename engine_type::is_dense;
    using is_rectangular   = typename engine_type::is_rectangular;
    using is_resizable     = std::false_type;
    using is_row_major     = std::conditional_t<engine_type::is_row_major::value,
                                                std::false_type,
                                                std::true_type>;

    using size_tuple       = typename engine_type::size_tuple;

public:
    matrix_transpose_engine();
    matrix_transpose_engine(engine_type const& eng);
    matrix_transpose_engine(matrix_transpose_engine&&);
    matrix_transpose_engine(matrix_transpose_engine const&);

    matrix_transpose_engine& operator =(matrix_transpose_engine&&);
    matrix_transpose_engine& operator =(matrix_transpose_engine const&);

    element_type      operator ()(size_t i) const;
    element_type      operator ()(size_t i, size_t j) const;
    element_type const* data() const noexcept;

    size_t            columns() const noexcept;
    size_t            rows() const noexcept;
    size_tuple        size() const noexcept;

    size_t            column_capacity() const noexcept;
    size_t            row_capacity() const noexcept;
    size_tuple        capacity() const noexcept;

private:
    engine_type*      mp_other;    //- For exposition; pointer to actual engine
};

```

§ 6.5. Engine Promotion Traits

Engine promotion traits are used by the arithmetic traits to determine the resulting engine types in an arithmetical expression.

§ 6.5.1. For Negation

Class template `matrix_engine_negate_promotion<Eng1>` implements a traits type that determines the resulting engine type when negating a *MathObj*.

```
template<class Eng1>
struct matrix_engine_negate_promotion
{
    using engine_type = ...;    //- Implementation-defined
};

template<class Eng1>
using matrix_engine_negate_t =
    typename matrix_engine_negate_promotion<Eng1>::engine_type;
```

§ 6.5.2. For Addition

Class template `matrix_engine_add_promotion<Eng1, Eng2>` implements a traits type that determines the resulting engine type when adding two compatible *MathObjs*.

```
template<class Eng1, class Eng2>
struct matrix_engine_add_promotion
{
    using engine_type = ...;    //- Implementation-defined
};

template<class Eng1, class Eng2>
using matrix_engine_add_t =
    typename matrix_engine_add_promotion<Eng1, Eng2>::engine_type;
```


§ 6.5.3. For Subtraction

Class template `matrix_engine_subtract_promotion<Eng1, Eng2>` implements a traits type that determines the resulting engine type when subtracting two compatible *MathObjs*.

```
template<class Eng1, class Eng2>
struct matrix_engine_subtract_promotion
{
    using engine_type = ...;    //- Implementation-defined
};

template<class Eng1, class Eng2>
using matrix_engine_subtract_t =
    typename matrix_engine_subtract_promotion<Eng1, Eng2>::engine_type;
```

§ 6.5.4. For Multiplication

Class template `matrix_engine_multiply_promotion<Eng1, Eng2>` implements a traits type that determines the resulting engine type when multiplying two compatible *MathObjs*.

```

template<class Eng1, class Eng2>
struct matrix_engine_multiply_promotion;
{
    using engine_type = ...;    //- Implementation-defined
};

//- A possible partial specialization
//
template<class T1, size_t Rows1, size_t Cols1, class T2, class Alloc2>
struct matrix_multiplication_engine_promotion<fs_matrix_engine<T1, Rows1, Cols1>,
                                              dr_matrix_engine<T2, Alloc2>>
{
    using element_type = matrix_element_promotion_t<T1, T2>;
    using alloc_type =
        typename std::allocator_traits<Alloc2>::template rebind_alloc<element_type>;
    using engine_type = dr_matrix_engine<element_type, alloc_type>;
};

template<class Eng1, class Eng2>
using matrix_engine_multiply_t =
    typename matrix_engine_multiply_promotion<Eng1, Eng2>::engine_type;

```

§ 6.6. Mathematical Types

This section describes the three main linear algebra object types proposed herein: the class templates `column_vector`, `row_vector`, and `matrix`. The purpose of these types is to model the corresponding mathematical concepts, manage underlying implementations through their engine type template parameter, and provide interfaces which can be employed by overloaded arithmetic operators.

§ 6.6.1. Helper Traits

Alias template `enable_if_resizable_t<Eng1, Eng2>` is a helper used by the *MathObj* types to manipulate overload resolution sets for member functions that perform dynamic storage management.

```
template<class Eng1, class Eng2>
using enable_if_resizable_t =
    typename std::enable_if_t<is_same_v<Eng1, Eng2> && Eng1::is_resizable_t
```

§ 6.6.2. Column Vector

Class template `column_vector<Eng, OpTraits>` represents a column vector, with element type and resource management implemented by the engine type `Eng`, and arithmetic operations specified by the operator traits type `OpTraits`. If the underlying engine type provides dynamic resizing, then this class will as well.

```

template<class Eng, class OpTraits>
class column_vector
{
public:
    using engine_type      = Eng;
    using element_type     = typename engine_type::element_type;
    using is_dense         = typename engine_type::is_dense;
    using is_rectangular   = typename engine_type::is_rectangular;
    using is_resizable     = typename engine_type::is_resizable;
    using size_tuple       = typename engine_type::size_tuple;
    using transpose_type    = row_vector<matrix_transpose_engine<engine_type>>
    using hermitian_type    = std::conditional_t<is_complex_v<element_type>,
                                                row_vector<Eng, OpTraitsF
                                                transpose_type>;

    static_assert(is_matrix_element_v<element_type>);

public:
    ~column_vector() = default;
    column_vector();
    column_vector(column_vector&&) = default;
    column_vector(column_vector const&) = default;
    template<class Eng2, class OpTraits2>
    column_vector(column_vector<Eng2, OpTraits2> const& src);

    template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
    column_vector(size_t rows);
    template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
    column_vector(size_t rows, size_t rowcap);

    column_vector& operator =(column_vector&&) = default;
    column_vector& operator =(column_vector const&) = default;
    template<class Eng2, class OpTraits2>
    column_vector& operator =(column_vector<Eng2, OpTraits2> const& rhs);

    //- Const element access.
    //
    element_type      operator ()(size_t i) const;
    element_type const* data() const noexcept;

    //- Accessors.
    //
    size_t            columns() const noexcept;
    size_t            rows() const noexcept;

```

```

size_tuple size() const noexcept;

size_t      column_capacity() const noexcept;
size_t      row_capacity() const noexcept;
size_tuple  capacity() const noexcept;

//- Transpose and Hermitian.
//
transpose_type t() const;
hermitian_type h() const;

//- Mutable element access.
//
element_type& operator()(size_t i);
element_type* data() noexcept;

//- Change capacity.
//
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void reserve(size_t rowcap);

//- Change size.
//
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void resize(size_t rows);

//- Change size and capacity in one shot.
//
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void resize(size_t rows, size_t rowcap);

//- Row operations.
//
void swap_rows(size_t i, size_t j);

private:
    template<class Eng2, class OpTraits2> friend class row_vector;

private:
    engine_type m_engine;

private:

```

```
column_vector(engine_type const& eng);  
};
```

§ 6.6.3. Row Vector

Class template `row_vector<Eng, OpTraits>` represents a row vector, with element type and resource management implemented by the engine type `Eng`, and arithmetic operations specified by the operator traits type `OpTraits`. If the underlying engine type provides dynamic resizing, then this class will as well.

```

template<class Eng, class OpTraits>
class row_vector
{
public:
    using engine_type      = Eng;
    using element_type     = typename engine_type::element_type;
    using is_dense         = typename engine_type::is_dense;
    using is_rectangular   = typename engine_type::is_rectangular;
    using is_resizable     = typename engine_type::is_resizable;
    using size_tuple       = typename engine_type::size_tuple;
    using transpose_type    = column_vector<matrix_transpose_engine<engine_type>,
                                           column_vector<Eng, OpTraits>,
                                           transpose_type>;
    static_assert(is_matrix_element_v<element_type>);

public:
    ~row_vector() = default;
    row_vector();
    row_vector(row_vector&&) = default;
    row_vector(row_vector const&) = default;
    template<class Eng2, class OpTraits2>
    row_vector(row_vector<Eng2, OpTraits2> const& src);
    template<class Eng2>
    row_vector(column_vector<matrix_transpose_engine<Eng2>> const& src);

    template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
    row_vector(size_t cols);
    template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
    row_vector(size_t cols, size_t colcap);

    row_vector&      operator =(row_vector&&) = default;
    row_vector&      operator =(row_vector const&) = default;
    template<class Eng2, class OpTraits2>
    row_vector&      operator =(row_vector<Eng2, OpTraits2> const& rhs);

    //- Const element access.
    //
    element_type      operator ()(size_t i) const;
    element_type const* data() const noexcept;

    //- Accessors.
    //

```

```

size_t      columns() const noexcept;
size_t      rows() const noexcept;
size_tuple  size() const noexcept;

size_t      column_capacity() const noexcept;
size_t      row_capacity() const noexcept;
size_tuple  capacity() const noexcept;

//- Transpose and Hermitian.
//
transpose_type  t() const;
hermitian_type  h() const;

//- Mutable element access.
//
element_type&   operator()(size_t i);
element_type*   data() noexcept;

//- Change capacity.
//
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void    reserve(size_t colcap);

//- Change size.
//
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void    resize(size_t cols);

//- Change size and capacity in one shot.
//
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void    resize(size_t cols, size_t colcap);

//- column operations.
//
void    swap_columns(size_t i, size_t j);

private:
    template<class Eng2, class OpTraits2> friend class column_vector;

private:
    engine_type    m_engine;

```



```
private:
    row_vector(engine_type const& eng);
};
```

§ 6.6.4. Matrix

Class template `matrix<Eng, OpTraits>` represents a matrix, with element type and resource management implemented by the engine type `Eng`, and arithmetic operations specified by the operator traits type `OpTraits`. If the underlying engine type provides dynamic resizing, then this class will as well.

```

template<class Eng, class OpTraits>
class matrix
{
public:
    using engine_type      = Eng;
    using element_type     = typename engine_type::element_type;
    using is_dense         = typename engine_type::is_dense;
    using is_rectangular   = typename engine_type::is_rectangular;
    using is_resizable     = typename engine_type::is_resizable;
    using size_tuple       = typename engine_type::size_tuple;
    using transpose_type    = matrix<matrix_transpose_engine<engine_type>>;
    using hermitian_type    = std::conditional_t<is_complex_v<element_type>,
                                                matrix<Eng, OpTraitsR>,
                                                transpose_type>;

    static_assert(is_matrix_element_v<element_type>);

public:
    ~matrix() = default;
    matrix();
    matrix(matrix&&) = default;
    matrix(matrix const&) = default;
    template<class Eng2, class OpTraits2>
    matrix(matrix<Eng2, OpTraits2> const& src);

    template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
    matrix(size_tuple size);
    template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
    matrix(size_t rows, size_t cols);

    template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
    matrix(size_tuple size, size_tuple cap);
    template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
    matrix(size_t rows, size_t cols, size_t rowcap, size_t colcap);

    matrix&      operator =(matrix&&) = default;
    matrix&      operator =(matrix const&) = default;
    template<class Eng2, class OpTraits2>
    matrix&      operator =(matrix<Eng2, OpTraits2> const& rhs);

    //- Const element access.
    //
    element_type      operator ()(size_t i, size_t j) const;
    element_type const* data() const noexcept;

```

```

//- Accessors.
//
size_t      columns() const noexcept;
size_t      rows() const noexcept;
size_tuple  size() const noexcept;

size_t      column_capacity() const noexcept;
size_t      row_capacity() const noexcept;
size_tuple  capacity() const noexcept;

//- Transpose and Hermitian.
//
transpose_type  t() const;
hermitian_type  h() const;

//- Mutable element access.
//
element_type&   operator()(size_t i, size_t j);
element_type*   data() noexcept;

//- Change capacity.
//
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void reserve(size_tuple cap);
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void reserve(size_t rowcap, size_t colcap);

//- Change size.
//
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void resize(size_tuple size);
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void resize(size_t rows, size_t cols);

//- Change size and capacity in one shot.
//
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void resize(size_tuple size, size_tuple cap);
template<class Eng2 = Eng, enable_if_resizable_t<Eng, Eng2> = true>
void resize(size_t rows, size_t cols, size_t rowcap, size_t colcap);

//- Row and column operations.

```

```

//
void    swap_columns(size_t i, size_t j);
void    swap_rows(size_t i, size_t j);

private:
    template<class Eng2, class OpTraits2> friend class matrix;

private:
    engine_type      m_engine;

private:
    matrix(engine_type const& eng);
};

```

§ 6.7. Matrix Arithmetic Traits

This section defines a set of arithmetic traits types for negation, addition, subtraction, and multiplication. The purpose of these traits types is threefold:

1. to determine the element type of the resulting *MathObj*;
2. to determine the engine type of the resulting *MathObj*; and
3. to carry out the arithmetical operation.

The idea here is that arithmetic operators (described below) simply forward to the appropriate traits type, which does the heavy lifting.

§ 6.7.1. Negation Traits

Class template `matrix_negation_traits<Opnd1, OpTraitsR>` is an arithmetic traits type that performs the negation of a *MathObj* and returns the result in another *MathObj* having an implementation-defined engine type. There are three partial specializations to support the three overloaded unary negation operators described below.

```

template<class Opnd1, class OpTraitsR>
struct matrix_negation_traits;

//- A possible partial specialization
//
template<class Eng1, class OpTraits1, class OpTraitsR>
struct matrix_negation_traits<matrix<Eng1, OpTraits1>, OpTraitsR>
{
    using engine_type = matrix_negation_engine_t<Eng1>;
    using op_traits    = OpTraitsR;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type negate(matrix<Eng1, OpTraits1> const& m1);
};

//- Other partial specializations
//
template<class Eng1, class OpTraits1, class OpTraitsR>
struct matrix_negation_traits<column_vector<Eng1, OpTraits1>, OpTraitsR>
{...};

template<class Eng1, class OpTraits1, class OpTraitsR>
struct matrix_negation_traits<row_vector<Eng1, OpTraits1>, OpTraitsR>
{...};

```

§ 6.7.2. Addition Traits

Class template `matrix_addition_traits<Opnd1, Opnd2, OpTraitsR>` is an arithmetic traits type that performs the addition of two compatible *MathObjs* and returns the result in a *MathObj* having an implementation-defined engine type. There are three partial specializations to support the three overloaded binary addition operators described below.

```

template<class Opnd1, class Opnd2, class OpTraitsR>
struct matrix_addition_traits;

//- A possible partial specialization
//
template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class Op>
struct matrix_addition_traits<matrix<Eng1, OpTraits1>,
                             matrix<Eng2, OpTraits2>,
                             OpTraitsR>
{
    using engine_type = matrix_addition_engine_t<Eng1, Eng2>;
    using op_traits    = OpTraitsR;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type add(matrix<Eng1, OpTraits1> const& m1,
                           matrix<Eng2, OpTraits2> const& m2);
};

//- Other partial specializations
//
template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class Op>
struct matrix_addition_traits<column_vector<Eng1, OpTraits1>,
                              column_vector<Eng2, OpTraits2>,
                              OpTraitsR>
{...};

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class Op>
struct matrix_addition_traits<row_vector<Eng1, OpTraits1>,
                              row_vector<Eng2, OpTraits2>,
                              OpTraitsR>
{...};

```

§ 6.7.3. Subtraction Traits

Class template `matrix_subtraction_traits<Opnd1, Opnd2, OpTraitsR>` is an arithmetic traits type that performs the subtraction of two compatible *MathObjs* and returns the result in a *MathObj* having an implementation-defined engine type. There are three partial specializations to support the three overloaded binary subtraction operators described below.

```

template<class Opnd1, class Opnd2, class OpTraitsR>
struct matrix_subtraction_traits;

//- A possible partial specialization
//
template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_subtraction_traits<matrix<Eng1, OpTraits1>,
                                matrix<Eng2, OpTraits2>,
                                OpTraitsR>
{
    using engine_type = matrix_subtraction_engine_t<Eng1, Eng2>;
    using op_traits    = OpTraitsR;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type subtract(matrix<Eng1, OpTraits1> const& m1,
                                matrix<Eng2, OpTraits2> const& m2);
};

//- Other partial specializations
//
template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_subtraction_traits<column_vector<Eng1, OpTraits1>,
                                column_vector<Eng2, OpTraits2>,
                                OpTraitsR>
{...};

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_subtraction_traits<row_vector<Eng1, OpTraits1>,
                                row_vector<Eng2, OpTraits2>,
                                OpTraitsR>
{...};

```

§ 6.7.4. Multiplication Traits

Class template `matrix_multiplication_traits<Opnd1, Opnd2, OpTraitsR>` is an arithmetic traits type that performs the multiplication of two compatible *MathObjs* and returns the result in a *MathObj* having an implementation-defined engine type. Note that there are thirteen partial specializations to support the thirteen overloaded binary multiplication operators described below.

```

template<class Opnd1, class Opnd2, class OpTraitsR>
struct matrix_multiplication_traits;

//- A possible partial specialization
//
template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<matrix<Eng1, OpTraits1>,
                                   matrix<Eng2, OpTraits2>,
                                   OpTraitsR>
{
    using engine_type = matrix_multiplication_engine_t<Eng1, Eng2>;
    using op_traits    = OpTraitsR;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type multiply(matrix<Eng1, OpTraits1> const& m1,
                                matrix<Eng2, OpTraits2> const& m2);
};

//- Other partial specializations
//
template<class Eng1, class OpTraits1, class T2, class OpTraitsR>
struct matrix_multiplication_traits<column_vector<Eng1, OpTraits1>, T2, OpTraitsR>
{...};

template<class Eng1, class OpTraits1, class T2, class OpTraitsR>
struct matrix_multiplication_traits<row_vector<Eng1, OpTraits1>, T2, OpTraitsR>
{...};

template<class T1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<T1, column_vector<Eng2, OpTraits2>, OpTraitsR>
{...};

template<class T1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<T1, row_vector<Eng2, OpTraits2>, OpTraitsR>
{...};

template<class Eng1, class OpTraits1, class T2, class OpTraitsR>
struct matrix_multiplication_traits<matrix<Eng1, OpTraits1>, T2, OpTraitsR>
{...};

template<class T1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<T1, matrix<Eng2, OpTraits2>, OpTraitsR>
{...};

```



```

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<column_vector<Eng1, OpTraits1>,
                                   row_vector<Eng2, OpTraits2>,
                                   OpTraitsR>
{...};

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<row_vector<Eng1, OpTraits1>,
                                   column_vector<Eng2, OpTraits2>,
                                   OpTraitsR>
{...};

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<matrix<Eng1, OpTraits1>,
                                   column_vector<Eng2, OpTraits2>,
                                   OpTraitsR>
{...};

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<matrix<Eng1, OpTraits1>,
                                   row_vector<Eng2, OpTraits2>,
                                   OpTraitsR>
{...};

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<column_vector<Eng1, OpTraits1>,
                                   matrix<Eng2, OpTraits2>,
                                   OpTraitsR>
{...};

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<row_vector<Eng1, OpTraits1>,
                                   matrix<Eng2, OpTraits2>,
                                   OpTraitsR>
{...};

```

One of the partial specializations bears special mention, and that is the one corresponding to the inner product. In this particular case, the return value from `multiply()` is not a *MathObj*, but instead a promoted element type:

```

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class Op>
struct matrix_multiplication_traits<row_vector<Eng1, OpTraits1>,
                                   column_vector<Eng2, OpTraits2>,
                                   OpTraitsR>
{
    using elem_type_1 = typename row_vector<Eng1, OpTraits1>::element_type;
    using elem_type_2 = typename column_vector<Eng2, OpTraits2>::element_type;
    using engine_type = void;
    using result_type = matrix_element_promotion_t<elem_type_1, elem_type_2>;

    static result_type multiply(row_vector<Eng1, OpTraits1> const& rv1,
                               column_vector<Eng2, OpTraits2> const& cv2)
};

```

§ 6.8. Operator Traits

Class `default_matrix_operator_traits` is a traits-style template parameter to `column_vector`, `row_vector`, and `matrix`. Its purpose is to convey a suggested set of arithmetical operations to be associated with a given `MathObj`.

```

struct default_matrix_operator_traits
{
    template<class Opnd1, class OpTraitsR>
    using negation_traits = matrix_negation_traits<Opnd1, OpTraitsR>;

    template<class Opnd1, class Opnd2, class OpTraitsR>
    using addition_traits = matrix_addition_traits<Opnd1, Opnd2, OpTraitsR>;

    template<class Opnd1, class Opnd2, class OpTraitsR>
    using subtraction_traits = matrix_subtraction_traits<Opnd1, Opnd2, OpTraitsR>;

    template<class Opnd1, class Opnd2, class OpTraitsR>
    using multiplication_traits = matrix_multiplication_traits<Opnd1, Opnd2, OpTraitsR>;
};

```

§ 6.9. Operator Traits Promotion

Class template `matrix_operator_traits_promotion<T1, T2>` is used by the arithmetic operators to select a specific arithmetic traits type that will be used to perform an operation.

```
template<class T1, class T2>
struct matrix_operator_traits_promotion;

template<class T1>
struct matrix_operator_traits_promotion<T1, T1>
{
    using traits_type = T1;
};

template<class T1>
struct matrix_operator_traits_promotion<T1, default_matrix_operator_traits>
{
    using traits_type = T1;
};

template<class T1>
struct matrix_operator_traits_promotion<default_matrix_operator_traits, T1>
{
    using traits_type = T1;
};

template<>
struct matrix_operator_traits_promotion<default_matrix_operator_traits,
                                         default_matrix_operator_traits>
{
    using traits_type = default_matrix_operator_traits;
};
```

§ 6.10. Arithmetic Operators

The arithmetic operators provide syntax that mimics common mathematical notation, with computation executed by an arithmetic traits type specified by at least one of the operands' template parameters.

Readers will note that the return types of the overloaded operators described below are left unspecified. This is a deliberate choice so that implementers have the freedom to choose whatever default technique for evaluating expressions they desire; for example, by returning temporary objects, or by using expression templates, or perhaps by some other technique.

§ 6.10.1. Negation Operator

Three overloaded unary negation operators are provided to perform element-wise negation of a *MathObj* instance.

```
template<class Eng1, class OpTraits1>
auto    operator -(column_vector<Eng1, OpTraits1> const& cv1);

template<class Eng1, class OpTraits1>
auto    operator -(row_vector<Eng1, OpTraits1> const& rv1);

template<class Eng1, class OpTraits1>
auto    operator -(matrix<Eng1, OpTraits1> const& m1);
```

Note that this function template is equivalent to multiplying the argument by the scalar value of `static_cast<Eng1>(-1)`.

§ 6.10.2. Addition Operator

Three overloaded binary addition operators are provided to perform element-wise addition of two *MathObj* instances.

```
template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto    operator +(column_vector<Eng1, OpTraits1> const& cv1,
                  column_vector<Eng2, OpTraits2> const& cv2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto    operator +(row_vector<Eng1, OpTraits1> const& rv1,
                  row_vector<Eng2, OpTraits2> const& rv2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto    operator +(matrix<Eng1, OpTraits1> const& m1,
                  matrix<Eng2, OpTraits2> const& m2);
```

Specifically, this operator performs the addition of two `MathObj` instances of identical dimension.

§ 6.10.3. Subtraction Operator

Three overloaded binary subtraction operators are provided to perform element-wise subtraction of two *MathObj* instances.

```
template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto    operator -(column_vector<Eng1, OpTraits1> const& cv1,
                  column_vector<Eng2, OpTraits2> const& cv2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto    operator -(row_vector<Eng1, OpTraits1> const& rv1,
                  row_vector<Eng2, OpTraits2> const& rv2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto    operator -(matrix<Eng1, OpTraits1> const& m1,
                  matrix<Eng2, OpTraits2> const& m2);
```

Specifically, this operator performs the subtraction of two `MathObj` instances of identical dimension.

§ 6.10.4. Multiplication Operator

Thirteen overloaded binary multiplication operators are provided to perform appropriate multiplication of two *MathObj* instances.

```

template<class Eng1, class OpTraits1, class Scalar2>
auto operator *(column_vector<Eng1, OpTraits1> const& cv1, Scalar2 s2);

template<class Scalar1, class Eng2, class OpTraits2>
auto operator *(Scalar1 s1, column_vector<Eng2, OpTraits2> const& cv2);

template<class Eng1, class OpTraits1, class Scalar2>
auto operator *(row_vector<Eng1, OpTraits1> const& rv1, Scalar2 s2);

template<class Scalar1, class Eng2, class OpTraits2>
auto operator *(Scalar1 s1, row_vector<Eng2, OpTraits2> const& rv2);

template<class Eng1, class OpTraits1, class Scalar2>
auto operator *(matrix<Eng1, OpTraits1> const& m1, Scalar2 s2);

template<class Scalar1, class Eng2, class OpTraits2>
auto operator *(Scalar1 s1, matrix<Eng2, OpTraits2> const& m2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto operator *(row_vector<Eng1, OpTraits1> const& rv1,
                column_vector<Eng2, OpTraits2> const& cv2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto operator *(column_vector<Eng1, OpTraits1> const& cv1,
                row_vector<Eng2, OpTraits2> const& rv2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto operator *(matrix<Eng1, OpTraits1> const& m1,
                column_vector<Eng2, OpTraits2> const& cv2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto operator *(matrix<Eng1, OpTraits1> const& m1,
                row_vector<Eng2, OpTraits2> const& rv2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto operator *(column_vector<Eng1, OpTraits1> const& cv1,
                matrix<Eng2, OpTraits2> const& m2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
auto operator *(row_vector<Eng1, OpTraits1> const& rv1,
                matrix<Eng2, OpTraits2> const& m2);

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>

```

```
auto operator *(matrix<Eng1, OpTraits1> const& m1,
               matrix<Eng2, OpTraits2> const& m2);
```

The first six function templates,

- `operator *(column_vector<Eng1, OpTraits1> const& cv1, Scalar2 s2),`
- `operator *(Scalar1 s1, column_vector<Eng2, OpTraits2> const& cv2),`
- `operator *(row_vector<Eng1, OpTraits1> const& rv1, Scalar2 s2),`
- `operator *(Scalar1 s1, row_vector<Eng2, OpTraits2> const& rv2),`
- `operator *(matrix<Eng1, OpTraits1> const& m1, Scalar2 s2),` and
- `operator *(Scalar1 s1, matrix<Eng2, OpTraits2> const& m2)`

perform pre- and post-multiplication of a *MathObj* instance by a scalar value.

The next two function templates,

- `operator *(row_vector<Eng1, OpTraits1> const& rv1, column_vector<Eng2, OpTraits2> const& cv2)` and
- `operator *(column_vector<Eng1, OpTraits1> const& cv1, row_vector<Eng2, OpTraits2> const& rv2)`

perform the inner product and outer product, respectively. Although hidden by the `auto` return type, per the multiplication traits described above, the inner product returns a promoted scalar and the outer product returns a `matrix`, as expected.

The last five function templates,

- `operator *(matrix<Eng1, OpTraits1> const& m1, column_vector<Eng2, OpTraits2> const& cv2),`
- `operator *(matrix<Eng1, OpTraits1> const& m1, row_vector<Eng2, OpTraits2> const& rv2),`
- `operator *(column_vector<Eng1, OpTraits1> const& cv1, matrix<Eng2, OpTraits2> const& m2),`
- `operator *(row_vector<Eng1, OpTraits1> const& rv1, matrix<Eng2, OpTraits2> const& m2),` and
- `operator *(matrix<Eng1, OpTraits1> const& m1, matrix<Eng2, OpTraits2> const& m2)`

perform matrix multiplication of *MathObjs*.

The question may arise as to why thirteen multiplication operators are provided rather than the expected fifteen. Strictly speaking, it is certainly possible to specify multiplication operators for `row_vector<Eng1>*row_vector<Eng2>` and `column_vector<Eng1>*column_vector<Eng2>`, but these represent degenerate cases of vectors having one element, and so are not included in this revision.

§ 6.11. Type Aliases

Some type aliases provide useful shorthand symbols for the most common compositions of types.

```
// - Convenience aliases for dynamically-resizable MathObjs
//
template<class T, class A = std::allocator<T>>
using dyn_column_vector = column_vector<dr_matrix_engine<T, A>>;

template<class T, class A = std::allocator<T>>
using dyn_col_vector = dyn_column_vector<T, A>;

template<class T, class A = std::allocator<T>>
using dyn_row_vector = row_vector<dr_matrix_engine<T, A>>;

template<class T, class A = std::allocator<T>>
using dyn_matrix = matrix<dr_matrix_engine<T, A>>;

// - Convenience aliases for fixed-size MathObjs
//
template<class T, size_t Rows>
using fs_column_vector = column_vector<fs_matrix_engine<T, Rows, 1>>;

template<class T, size_t Rows>
using fs_col_vector = fs_column_vector<T, Rows>;

template<class T, size_t Cols>
using fs_row_vector = row_vector<fs_matrix_engine<T, 1, Cols>>;

template<class T, size_t Rows, size_t Cols>
using fs_matrix = matrix<fs_matrix_engine<T, Rows, Cols>>;
```


§ 6.12. Expression Results

The following are some expressions that demonstrate how element and engine promotions are performed.

```

using cx_float   = std::complex<float>;
using cx_double  = std::complex<double>;
using namespace std::la;

void t1()
{
    float      f = 1.0f;
    double     d = 1.0;
    cx_double   Cols = {1.0, 0.0};

    dyn_matrix<float>          dmf(3, 3);
    dyn_matrix<double>        dmd(3, 3);
    dyn_matrix<cx_double>     dmc(3, 3);

    fs_matrix<float, 3, 3>     fmf;
    fs_matrix<double, 3, 3>    fmd;
    fs_matrix<cx_double, 3, 3> fmc;

    auto    m01 = dmf*fmf;      //- dyn_matrix<float>
    auto    m02 = dmd*fmd;      //- dyn_matrix<double>
    auto    m03 = dmc*fmc;      //- dyn_matrix<complex<double>>

    auto    m04 = fmf*dmf;      //- dyn_matrix<float>
    auto    m05 = fmd*dmd;      //- dyn_matrix<double>
    auto    m06 = fmc*dmc;      //- dyn_matrix<complex<double>>

    auto    m07 = fmf*fmd;      //- fs_matrix<double, 3, 3>
    auto    m08 = fmd*fmc;      //- fs_matrix<complex<double>, 3, 3>
}

void t2()
{
    dyn_col_vector<float>      dcvf(3);
    dyn_col_vector<double>     dcvd(3);

    fs_col_vector<float, 3>    fcvf;
    fs_col_vector<double, 3>   fcvd;

    dyn_row_vector<float>      drvf(3);
    dyn_row_vector<double>     drvd(3);

    fs_row_vector<float, 3>    frvf;
    fs_row_vector<double, 3>   frvd;
}

```

```

dyn_matrix<double>      dmd(3, 3);
dyn_matrix<float>       dmf(3, 3);
dyn_matrix<float>       dmf_cv(3, 1);
dyn_matrix<float>       dmf_rv(1, 3);

fs_matrix<double, 3, 3>  fmd;
fs_matrix<float, 3, 3>   fmf;
fs_matrix<float, 3, 1>   fmf_cv;
fs_matrix<float, 1, 3>   fmf_rv;

auto    r01 = dmf * dcvf;      //- dyn_col_vector<float>
auto    r02 = dmf_cv * drvf;   //- dyn_matrix<float>
auto    r03 = drvf * dmf;      //- dyn_row_vector<float>
auto    r04 = dcvf * fmf_rv;   //- dyn_matrix<float>

auto    r11 = dmf * dcvd;      //- dyn_col_vector<double>
auto    r12 = dmf_cv * drvd;   //- dyn_matrix<double>
auto    r13 = drvf * dmd;      //- dyn_row_vector<double>
auto    r14 = dcvd * dmf_rv;   //- dyn_matrix<double>

auto    r21 = fmf * fcvf;      //- fs_col_vector<float, 3>
auto    r22 = fmf_cv * frvf;   //- fs_matrix<float, 3, 3>
auto    r23 = frvf * fmf;      //- fs_row_vector<float, 3>
auto    r24 = fcvf * fmf_rv;   //- fs_matrix<float, 3, 3>

auto    r31 = fmf * fcvd;      //- fs_col_vector<double, 3>
auto    r32 = fmf_cv * frvd;   //- fs_matrix<double, 3, 3>
auto    r33 = frvf * fmd;      //- fs_row_vector<3>
auto    r34 = fcvd * fmf_rv;   //- fs_matrix<double, 3, 3>

auto    r41 = drvf * dcvf;     //- float
auto    r42 = frvf * dcvd;     //- double
auto    r43 = frvd * fcvd;     //- double
}

```

§ 7. How Does it Work?

The preceding sections described the various components comprising the interface, starting with the most basic, and proceeding in a bottom-up fashion. While this is useful for understanding the static

structure of the interface, it may help to look at how the interface works from a top-down perspective in order to understand how the pieces work together.

Consider the following simple function:

```
void f()
{
    fs_matrix<double, 3, 4>    fsmd;        //- fixed-size matrix of double
    dyn_matrix<float>          drmf(4, 5);   //- dynamic matrix of float

    auto    m1 = fsmd * drmf;    //- dyn_matrix<double> of size 3x5
}
```

One might reasonably ask: how does the library determine that `m1` is a dynamically-resizable matrix of double, and how does it know how to perform the requested multiplication?

At a high level, the process for determining the return type of the binary operators is as follows:

1. An operator, say multiplication, is invoked.
2. The operator examines the operator traits types of its two operands, employing operator traits promotion to determine which of the two it will use to perform the multiplication; this operator traits type is also passed on as a template argument to the result type.
3. The operator extracts the appropriate arithmetic traits type from the operator traits.
4. The arithmetic traits examines the engine types of the two operands, employing engine promotion to determine the engine type of the result.
5. The engine promotion traits examines the element types of the two operands, employing element promotion to determine the element type of the result.
6. The element promotion traits determines a suitable element type that can contain the result of the arithmetic operation with little or no loss of information.
7. The engine promotion traits specifies the appropriate engine type parametrized in terms of the resulting element type and other types specific to that engine type.
8. The arithmetic traits combines the engine type with the operator traits type to specify the actual *MathObj* return type.
9. The arithmetic traits provides a static member function that implements the multiplication operation.
10. The multiplication operator calls that function with the supplied operands and returns the result.

§ 7.1. In More Detail

We describe the process in more detail in the following sections. The reader may find it helpful to follow along by looking at the strawman implementation provided in the repo listed at the top of this document.

§ 7.1.1. Calling an Operator

Let's begin by examining the instantiation of matrix `m1`, in which the binary multiplication operator is invoked with operands `fsmd` and `drmf`. The implementation of that operator might look like the following:

```
template<class Eng1, class OpTraits1, class Eng2, class OpTraits2>
inline auto
operator *(matrix<Eng1, OpTraits1> const& m1, matrix<Eng2, OpTraits2> const& m2)
{
    using op_traits = matrix_operator_traits_promotion_t<OpTraits1, OpTraits2>;
    using op1_type = matrix<Eng1, OpTraits1>;
    using op2_type = matrix<Eng2, OpTraits2>;
    using mul_traits =
        typename op_traits::template multiplication_traits<op1_type, op2_type>;

    return mul_traits::multiply(m1, m2);
}
```

The template parameters are deduced as:

```
Eng1    -->> fs_matrix_engine<double, 3, 4>
OpTraits1 -->> default_matrix_operator_traits
Eng2    -->> dr_matrix_engine<float, std::allocator<float>>
OpTraits2 -->> default_matrix_operator_traits
```

§ 7.1.2. Selecting the Operator Traits

The resulting operator traits type is then determined by instantiating the alias template `matrix_operator_traits_promotion_t` from Section 6.9 above:

```

op_traits  -->>  matrix_operator_traits_promotion_t<default_matrix_operator
               default_matrix_operator
               -->>  matrix_operator_traits_promotion<default_matrix_operator_t
               default_matrix_operator_t
               -->>  default_matrix_operator_traits

```

Now that the operator traits are known, we also know the four corresponding arithmetic traits templates that will determine the resulting engine type and perform the actual computational work.

The operand types are:

```

op1_type  -->>  matrix<fs_matrix_engine<double, 3, 4>,
               default_matrix_operator_traits>

op2_type  -->>  matrix<dr_matrix_engine<float, std::allocator<float>>,
               default_matrix_operator_traits>

```

§ 7.1.3. Selecting the Multiplication Traits

Given the operand types and operator traits types, the multiplication traits type is then determined using them, per Section 6.8:

```

mul_traits  -->>
    default_matrix_operator_traits::
        matrix_multiplication_traits<
            matrix<fs_matrix_engine<double, 3, 4>, default_matrix_operator_
            matrix<dr_matrix_engine<float, std::allocator<float>>,
            default_matrix_operator_traits>

```

From Section 6.7.4 above, we can see that the appropriate multiplication traits partial specialization is:

```

template<class Eng1, class OpTraits1, class Eng2, class OpTraits2, class OpTraitsR>
struct matrix_multiplication_traits<matrix<Eng1, OpTraits1>,
                                   matrix<Eng2, OpTraits2>,
                                   OpTraitsR>
{
    using engine_type = matrix_multiplication_engine_t<Eng1, Eng2>;
    using op_traits    = OpTraitsR;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type multiply(matrix<Eng1, OpTraits1> const& m1,
                                matrix<Eng2, OpTraits2> const& m2);
};

```

§ 7.1.4. Selecting the Engine Type

Now that we know the operator traits to be passed on to the resulting matrix, and which specialization will perform the multiplication, the next step in the process is to determine the engine type of the result. We know that the template arguments to the multiplication traits are:

```

Eng1      -->> fs_matrix_engine<double, 3, 4>
OpTraits1 -->> default_matrix_operator_traits
Eng2      -->> dr_matrix_engine<float, std::allocator<float>>
OpTraits2 -->> default_matrix_operator_traits
OpTraitsR -->> default_matrix_operator_traits

```

We use these to select the resulting engine type. The instantiation of the alias template `matrix_multiplication_engine_t` takes us to the partial specialization of engine promotion traits relevant to multiplication:

```

engine_type -->> matrix_multiplication_engine_t<
                  fs_matrix_engine<double, 3, 4>,
                  dr_matrix_engine<float, std::allocator<float>>>
-->> matrix_multiplication_engine_promotion<
      fs_matrix_engine<double, 3, 4>,
      dr_matrix_engine<float, std::allocator<float>>>::eng

```

From the example in Section 6.5.4 above, this partial specialization looks like the following:

```
template<class T1, size_t Rows1, size_t Cols1, class T2, class Alloc2>
struct matrix_multiplication_engine_promotion<fs_matrix_engine<T1, Rows1, Cols1>,
                                              dr_matrix_engine<T2, Alloc2>>
{
    using element_type = matrix_element_promotion_t<T1, T2>;
    using alloc_type =
        typename std::allocator_traits<Alloc2>::template rebind_alloc<element_type>;
    using engine_type = dr_matrix_engine<element_type, alloc_type>;
};
```

We can see from the above that this example engine promotion traits will specify a dynamically resizable engine as the result type. The template arguments are:

```
T1      -->> double
Rows1   -->> 3
Cols1   -->> 4
T2      -->> float
Alloc2  -->> std::allocator<float>
```

From these, we can determine the promoted element type. The instantiation of template alias `matrix_element_promotion_t` leads to:

```
element_type -->> matrix_element_promotion_t<double, float>
              -->> matrix_element_promotion<double, float>::type
              -->> double
```

We next use the rebinder facility from `std::allocator_traits` to select the resulting allocator type:

```
alloc_type -->> std::allocator_traits<std::allocator<float>>::rebind_alloc<double>
              -->> std::allocator<double>
```

These are then combined to yield the resulting engine type:

```
engine_type -->> dr_matrix_engine<double, std::allocator<double>>
```


Now that we know the engine type, it is possible to determine the template arguments of the resulting matrix type. Popping the instantiation stack and returning to `matrix_multiplication_traits`, we now know that:

```
engine_type  -->  dr_matrix_engine<double, std::allocator<double>>
op_traits    -->  default_matrix_operator_traits
```

These now form the template arguments of resulting matrix type:

```
result_type  -->  matrix<dr_matrix_engine<double, std::allocator<double>>,
                    default_matrix_operator_traits>
```

§ 7.1.5. The Fully Instantiated Multiplication Traits

So, after all of the preceding instantiations are done, the fully specialized matrix multiplication traits looks like this:

```
template<>
struct matrix_multiplication_traits<
    matrix<fs_matrix_engine<double, 3, 4>, default_matrix_operator_traits>,
    matrix<dr_matrix_engine<float, std::allocator<float>>,
    default_matrix_operator_traits>
{
    using engine_type = dr_matrix_engine<double, std::allocator<double>>;
    using op_traits    = default_matrix_operator_traits;
    using result_type  = matrix<dr_matrix_engine<double, std::allocator<double>>,
                                default_matrix_operator_traits>;

    static
    matrix<dr_matrix_engine<double, std::allocator<double>>, default_matrix_operator_traits>
    multiply(matrix<fs_matrix_engine<double, 3, 4>, default_matrix_operator_traits>,
            matrix<dr_matrix_engine<float, std::allocator<float>>, default_matrix_operator_traits> const&);
};
```

(NB: this not a user-supplied specialization; rather, it is intended to represent what the compiler "sees" internally.)

Finally, the binary multiplication operator wraps the call to `multiply()`, returning the result as determined above.

§ 8. Customization

The library provides for three forms of customization: custom element types, custom engines, and custom arithmetical operations. The following sections show examples of each.

§ 8.1. Integrating a New Element Type

Suppose that you have created a new type that models a real number in some way and you wish for that type to be used as a matrix element:

```

class NewNum
{
public:
    NewNum();
    NewNum(NewNum&&) = default;
    NewNum(NewNum const&) = default;
    template<class U>    NewNum(U other);

    NewNum&    operator =(NewNum&&) = default;
    NewNum&    operator =(NewNum const&) = default;
    template<class U>    NewNum&    operator =(U rhs);

    NewNum    operator -() const;
    NewNum    operator +() const;

    NewNum&    operator +=(NewNum rhs);
    NewNum&    operator -=(NewNum rhs);
    NewNum&    operator *=(NewNum rhs);
    NewNum&    operator /=(NewNum rhs);

    template<class U>    NewNum&    operator +=(U rhs);
    template<class U>    NewNum&    operator -=(U rhs);
    template<class U>    NewNum&    operator *=(U rhs);
    template<class U>    NewNum&    operator /=(U rhs);

private:
    ...    //- implementation stuff
};

inline bool operator ==(NewNum lhs, NewNum rhs);
template<class U> inline bool operator ==(NewNum lhs, U rhs);
template<class U> inline bool operator ==(U lhs, NewNum rhs);
...
//- other comparison operators...
...
//- other arithmetic operators...
...
inline bool operator *(NewNum lhs, NewNum rhs);
template<class U> inline bool operator *(NewNum lhs, U rhs);
template<class U> inline bool operator *(U lhs, NewNum rhs);

```

Assuming that this type works as intended, and that all arithmetic interactions with other types are handled the set of operator overloads that you provide, then all that is required for the library to accept `NewNum` as an element type is this:

```
template<>
struct std::is_matrix_element<NewNum> : public std::true_type
{};
```

By stating that the type is a matrix element type, and by ensuring that its arithmetic operators functions as promised, the library's traits types will allow compilation to succeed.

However, if the default arithmetic traits do not understand how to use your new element type, then you will need to provide specializations of them as well.

§ 8.2. Integrating a New Engine Type

Suppose that you want to create a custom fixed-size matrix engine that is somehow different from `fs_matrix_engine`; perhaps it is instrumented in some way for debugging, or uses fixed-size storage that is external to the engine object. It might look like this:

```

template<class T, size_t Rows, size_t Cols>
class fs_matrix_engine_TST
{
    static_assert(Rows >= 1);
    static_assert(Cols >= 1);

public:
    using element_type    = T;
    using is_dense        = std::true_type;
    using is_rectangular  = std::true_type;
    using is_resizable    = std::false_type;
    using is_row_major    = std::true_type;
    using size_tuple      = std::tuple<size_t, size_t>;

public:
    fs_matrix_engine_TST() = default;
    fs_matrix_engine_TST(fs_matrix_engine_TST&&) = default;
    fs_matrix_engine_TST(fs_matrix_engine_TST const&) = default;

    fs_matrix_engine_TST& operator =(fs_matrix_engine_TST&&) = default;
    fs_matrix_engine_TST& operator =(fs_matrix_engine_TST const&) = default;

    T          operator ()(size_t i) const;
    T          operator ()(size_t i, size_t j) const;
    T const*   data() const noexcept;

    size_t     columns() const noexcept;
    size_t     rows() const noexcept;
    size_tuple size() const noexcept;

    size_t     column_capacity() const noexcept;
    size_t     row_capacity() const noexcept;
    size_tuple capacity() const noexcept;

    T&         operator ()(size_t i);
    T&         operator ()(size_t i, size_t j);
    T*         data() noexcept;

    void        swap_columns(size_t i, size_t j);
    void        swap_rows(size_t i, size_t j);

private:

```

```
...    //- implementation stuff
};
```

For each arithmetic operation in which you expect the new engine type to be involved, you will need to provide a specialization (full or partial) of the engine promotion traits for that operation. For example, let's assume that you're only interested in addition operations involving two operands having the new engine type, or where one operand has the standard fixed-size engine and the other has the new engine. Then your engine promotion traits might look like this:

```
template<class T1, size_t Rows1, size_t Cols1, class T2, size_t Rows2, size_t Cols2>
struct matrix_addition_engine_promotion<fs_matrix_engine_TST<T1, Rows1, Cols1>,
                                         fs_matrix_engine_TST<T2, Rows2, Cols2>>
{
    static_assert(Rows1 == Rows2);
    static_assert(Cols1 == Cols2);
    using element_type = matrix_element_promotion_t<T1, T2>;
    using engine_type  = fs_matrix_engine_TST<element_type, Rows1, Cols1>;
};

template<class T1, size_t Rows1, size_t Cols1, class T2, size_t Rows2, size_t Cols2>
struct matrix_addition_engine_promotion<fs_matrix_engine<T1, Rows1, Cols1>,
                                         fs_matrix_engine_TST<T2, Rows2, Cols2>>
{
    static_assert(Rows1 == Rows2);
    static_assert(Cols1 == Cols2);
    using element_type = matrix_element_promotion_t<T1, T2>;
    using engine_type  = fs_matrix_engine_TST<element_type, Rows1, Cols1>;
};

template<class T1, size_t Rows1, size_t Cols1, class T2, size_t Rows2, size_t Cols2>
struct matrix_addition_engine_promotion<fs_matrix_engine_TST<T1, Rows1, Cols1>,
                                         fs_matrix_engine<T2, Rows2, Cols2>>
{
    static_assert(Rows1 == Rows2);
    static_assert(Cols1 == Cols2);
    using element_type = matrix_element_promotion_t<T1, T2>;
    using engine_type  = fs_matrix_engine_TST<element_type, Rows1, Cols1>;
};
```

As we can see, these promotion traits dictate the result engine type to be the new engine type. Another option is that the result engine type be `fs_matrix_engine`; yet another option is that the return engine type be some other type, like an expression engine.

What about performing the computation? If the default implementation of the addition arithmetic traits understands how to perform the addition operation, then you are done.

However, if the default addition traits don't understand how to use the new engine type, then you will also have to write specializations of the corresponding addition traits types. That might look something like this:

```

template<class T1, size_t Rows, size_t Cols, class T2, class OpTraitsR>
struct matrix_addition_traits<matrix<fs_matrix_engine_TST<T1, Rows, Cols>,
                                matrix<fs_matrix_engine_TST<T2, Rows, Cols>,
                                OpTraitsR>
{
    using engine_type = matrix_addition_engine_t<fs_matrix_engine_TST<T1, Rows, Cols>,
                                                fs_matrix_engine_TST<T2, Rows, Cols>,
                                                OpTraitsR>;

    using op_traits    = OpTraitsR;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type add(matrix<fs_matrix_engine_TST<T1, Rows, Cols>, OpTraitsR,
                           matrix<fs_matrix_engine_TST<T2, Rows, Cols>, OpTraitsR);
};

```

```

template<class T1, size_t Rows, size_t Cols, class T2, class OpTraitsR>
struct matrix_addition_traits<matrix<fs_matrix_engine<T1, Rows, Cols>, OpTraitsR,
                                matrix<fs_matrix_engine_TST<T2, Rows, Cols>,
                                OpTraitsR>
{
    using engine_type = matrix_addition_engine_t<fs_matrix_engine<T1, Rows, Cols>,
                                                fs_matrix_engine_TST<T2, Rows, Cols>,
                                                OpTraitsR>;

    using op_traits    = OpTraitsR;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type add(matrix<fs_matrix_engine<T1, Rows, Cols>, OpTraitsR,
                           matrix<fs_matrix_engine_TST<T2, Rows, Cols>, OpTraitsR);
};

```

```

template<class T1, size_t Rows, size_t Cols, class T2, class OpTraitsR>
struct matrix_addition_traits<matrix<fs_matrix_engine_TST<T1, Rows, Cols>,
                                matrix<fs_matrix_engine<T2, Rows, Cols>, OpTraitsR>,
                                OpTraitsR>
{
    using engine_type = matrix_addition_engine_t<fs_matrix_engine_TST<T1, Rows, Cols>,
                                                fs_matrix_engine<T2, Rows, Cols>,
                                                OpTraitsR>;

    using op_traits    = OpTraitsR;
    using result_type  = matrix<engine_type, op_traits>;

    static result_type add(matrix<fs_matrix_engine_TST<T1, Rows, Cols>, OpTraitsR,
                           matrix<fs_matrix_engine_TST<T2, Rows, Cols>, OpTraitsR);
};

```


Note that namespace qualifiers which should actually appear in the above code have been omitted in the interest of brevity.

§ 8.3. Customizing an Arithmetic Operation

Suppose that you are working on a game engine, and you are interested in experimenting with different algorithms for performing affine transforms on spatial vectors. To do this, you might be using fixed-size 4 x 4 matrices and fixed size column vectors of size 4 x 1. To test your new algorithm(s), you could specialize the multiplication traits, which might look like this:

```
template<>
struct matrix_multiplication_traits<fs_matrix<float, 4, 4>,
                                   fs_column_vector<float, 4>,
                                   default_matrix_operator_traits>
{
    using engine_type = matrix_multiplication_engine_t<fs_matrix<float, 4, 4>,
                                                       fs_column_vector<float, 4>,
                                                       default_matrix_operator_traits>;

    using result_type = column_vector<engine_type, default_matrix_operator_traits>;

    static result_type multiply(fs_matrix<float, 4, 4> const& m1,
                               fs_column_vector<float, 4> const& cv2);
};
```

With this specialization visible, the following code would call the new, specialized `multiply()` member function, and thus employ your new algorithm:

```
void f()
{
    fs_matrix<float, 4, 4>    m1;
    fs_column_vector<float, 4> cv1, cvr;

    cvr = m1 * cv1;
}
```

§ 9. Future Directions

§ 9.1. Kona Feedback

At the Kona 2019 meeting, draft version D1 of this paper was reviewed by LEWG(I) and a joint session of SG14 and SG19. Both reviews were generally positive, several good suggestions were made, and some polls regarding future directions were taken.

§ 9.1.1. LEWG(I) Polls and Feedback (Wednesday 2019-02-20)

There were a few polls taken in this session:

1. We want 0-based indexing as opposed to 1-based indexing.
(unanimous: 20)
2. We like having separate `row_vector` and `column_vector` types in addition to `matrix`.
SF F N A SA (21 present)
3 0 5 4 4
3. We want explicitly named operations (e.g., `dot` and `outer`) in addition to operators.
SF F N A SA (21 present)
8 5 2 1 0
4. Define engine/matrix classes in terms of `mdspan` + storage and `mdspan` concepts (e.g., `extents`), and expose an `mdspan`-esque interface. This implies that `fs_` and `dyn_` are combined into one template parametrized on `extents` (which are either static or dynamic).
SF F N A SA (22 present)
6 2 7 0 0

There were some additional requests:

- Provide some implementation and usage experience.
- Provide a comparison with prior art.
- Explore the re-usability of `mdspan` and `common_type`.
- Be careful of allowing specializations of traits types that are part of namespace `std`; be consistent with other traits.

§ 9.1.2. Feedback From Joint SG14/SG19 Session (Friday 2019-02-22)

- Stick to 0-based indexing, for compatibility with current practice, and also for performance reasons.
- Provide a fixed-size engine whose memory is dynamically allocated.
- In this session, there was broad agreement that the one-vector approach advocated by LEWG(I) was the way to proceed.
- Outer product computation is rare in practice, therefore the vector-vector multiplication operator should return the inner product, and the outer product should be a named function.

§ 9.1.3. Other Suggestions Gathered Received at Kona

- Figure out how to incorporate executors for concurrent operations.
- Per SG20's recent guidance, include a "Gor table" (for example, see [P1362R0](#), Section 4.4) showing feature levels and anticipated user sophistication for each.
- Include tutorial material on how the library can be used and extended, with several illustrative examples.